STOCKIQ · DOCUMENTAÇÃO DO BACKEND

Backend API REST

Documentação técnica da API do StockIQ: arquitetura, modelo de dados, regras de negócio, segurança, mensageria, armazenamento e catálogo completo de endpoints.

Node.js · Express | TypeScript | PostgreSQL · Prisma | JWT · Zod | Amazon S3 · SQS | 88 endpoints

Gabriel da Silveira Pessoni · Lívia Portela Ferreira
FATEC Franca — Desenvolvimento de Software Multiplataforma · 6º semestre

Projeto Integrador VI
Computação em Nuvem II · Mineração de Dados

Sumário

01	**Visão geral**	Escopo, números e responsabilidades
02	**Stack tecnológica**	Tecnologias e finalidade
03	**Arquitetura em camadas**	Rotas, controllers, services, Prisma
04	**Ciclo de vida da requisição**	Pipeline de middlewares
05	**Modelo de dados**	ER, entidades e enumerações
06	**Autenticação e autorização**	JWT, RBAC e matriz de permissões
07	**Regras de negócio**	Movimentações, inventário, fotos, analytics
08	**Catálogo de endpoints**	17 módulos · 88 rotas
09	**Validação e tratamento de erros**	Zod, AppError, códigos HTTP
10	**Segurança e auditoria**	Helmet, CORS, rate limit, audit log
11	**Testes automatizados**	173 testes · 22 suítes
12	**Operação e execução**	Variáveis, scripts, Swagger
13	**Infraestrutura em nuvem**	S3, SQS e implantação-alvo

01 Visão geral

O backend do StockIQ é uma API REST em Node.js/Express com TypeScript que concentra toda a regra de negócio, a persistência em PostgreSQL (via Prisma ORM) e a autenticação do sistema. É consumida pelo front-end Next.js e pelo aplicativo mobile por HTTP/JSON.

17	88	14	173
MÓDULOS	ENDPOINTS	ENTIDADES	TESTES

A API cobre o ciclo completo de operação de um armazém: produtos (com foto no S3), categorias, marcas, fornecedores e clientes; endereçamento físico; movimentações de estoque; lotes com validade; contagens de inventário com **ajuste de estoque assíncrono por fila (SQS)**; alertas; auditoria; dashboard, relatórios e analytics (previsão de demanda e matriz ABC×XYZ); e configurações do sistema.

🔐
Segurança
JWT, bcrypt, Helmet, CORS, rate limiting e trilha de auditoria.

✔
Validação
Schemas Zod por módulo antes de qualquer regra de negócio.

⚙
Camadas claras
routes → controllers → services → Prisma.

☁
Nuvem
Fotos no Amazon S3 e fila de inventário no Amazon SQS, com worker dedicado.

📄
Documentada
Swagger UI e OpenAPI 3.0 gerados das rotas (`/docs`).

🧪
Testada
173 testes (Jest + Supertest) contra PostgreSQL real.

02 Stack tecnológica

Camada	Tecnologia	Finalidade
Linguagem	TypeScript 5	Tipagem estática ponta a ponta
Framework HTTP	Express 4	Roteamento e cadeia de middlewares
ORM	Prisma 6	Modelagem, migrations versionadas e acesso tipado ao PostgreSQL
Banco de dados	PostgreSQL 16	Persistência relacional transacional
Autenticação	jsonwebtoken · bcryptjs	Token JWT assinado e hash de senha (custo 10)
Validação	Zod 3	Schemas de entrada com mensagens padronizadas

Camada	Tecnologia	Finalidade
Segurança	Helmet · CORS · express-rate-limit	Cabeçalhos seguros, origem e limite de requisições
Upload	multer 2	Recepção de multipart em memória, com limite de tamanho e tipo
Nuvem	AWS SDK v3 (S3, SQS)	Armazenamento de fotos e mensageria assíncrona
Documentação	swagger-jsdoc · swagger-ui-express	OpenAPI 3.0 em `/docs` e `/docs.json`
Testes	Jest 29 · Supertest · ts-jest	Testes unitários e de integração
Logs	Morgan	Log de requisições HTTP (silenciado nos testes)

03 Arquitetura em camadas

Cada módulo de negócio é composto por artefatos com responsabilidades distintas. Preocupações transversais (autenticação, validação, auditoria, limites, upload e erros) vivem em middlewares e nunca dentro de controllers ou services.

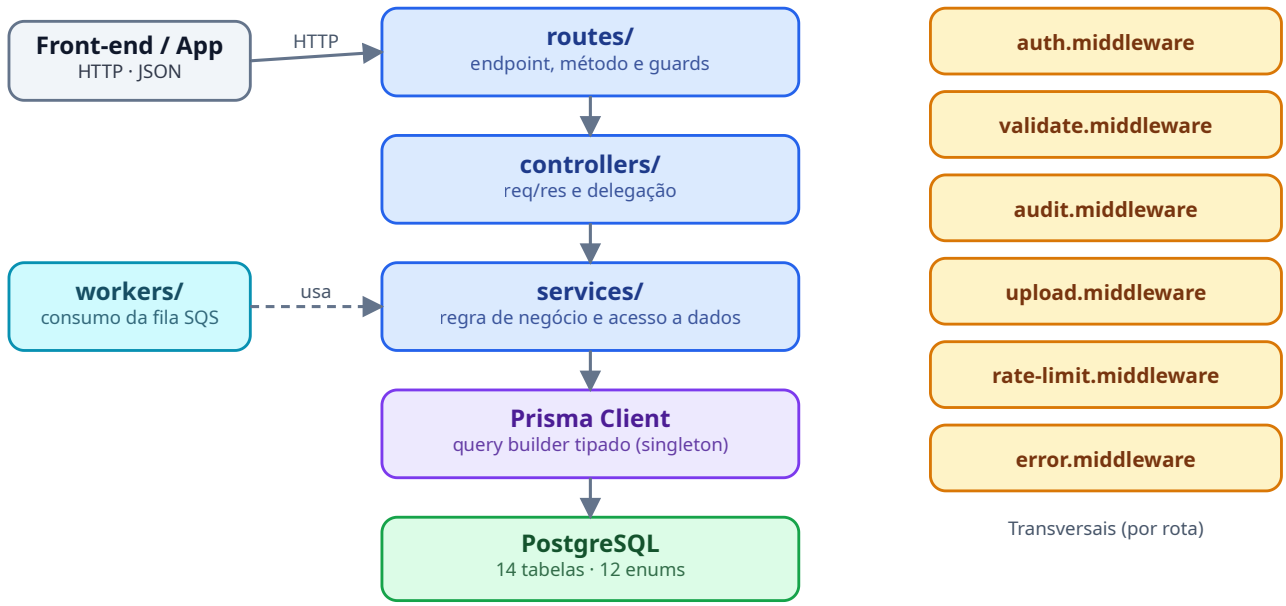


Figura 1 — Camadas da API, middlewares transversais e worker

- **Estrutura de pastas**

```
backend/
├── src/
│   ├── app.ts                  # Express, middlewares globais e registro de rotas
│   ├── server.ts               # Bootstrap: banco, HTTP e worker embutido
│   ├── worker.ts               # Processo dedicado ao consumo das filas (npm run worker)
│   ├── config/swagger.ts       # Definição OpenAPI
│   ├── routes/         (18)    # Endpoints + guards de acesso
│   ├── controllers/    (13)    # req/res
│   ├── services/       (20)    # Regra de negócio, Prisma, storage (S3) e fila (SQS)
│   ├── schemas/        (10)    # Schemas Zod de entrada
│   ├── middleware/     (6)     # auth · validate · audit · upload · error · rate-limit
│   ├── workers/                # inventory.worker.ts
│   └── prisma/client.ts        # Prisma Client singleton
├── prisma/                     # schema.prisma · seed.ts · migrations/ (5)
├── scripts/                    # setup-queues · cleanup-storage · export-swagger · check-swagger
├── __tests__/                  # Testes (Jest + Supertest) — 22 suítes
└── .env.example
```

04 Ciclo de vida da requisição

Toda requisição atravessa uma cadeia fixa de middlewares. Qualquer falha é convertida em uma resposta JSON padronizada pelo manipulador de erros central.

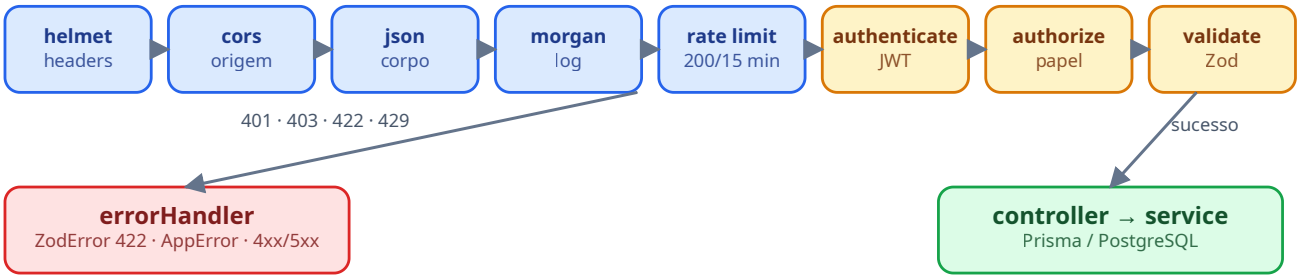


Figura 2 — Pipeline de middlewares (azul: globais · âmbar: por rota)

Sequência — registro de uma movimentação

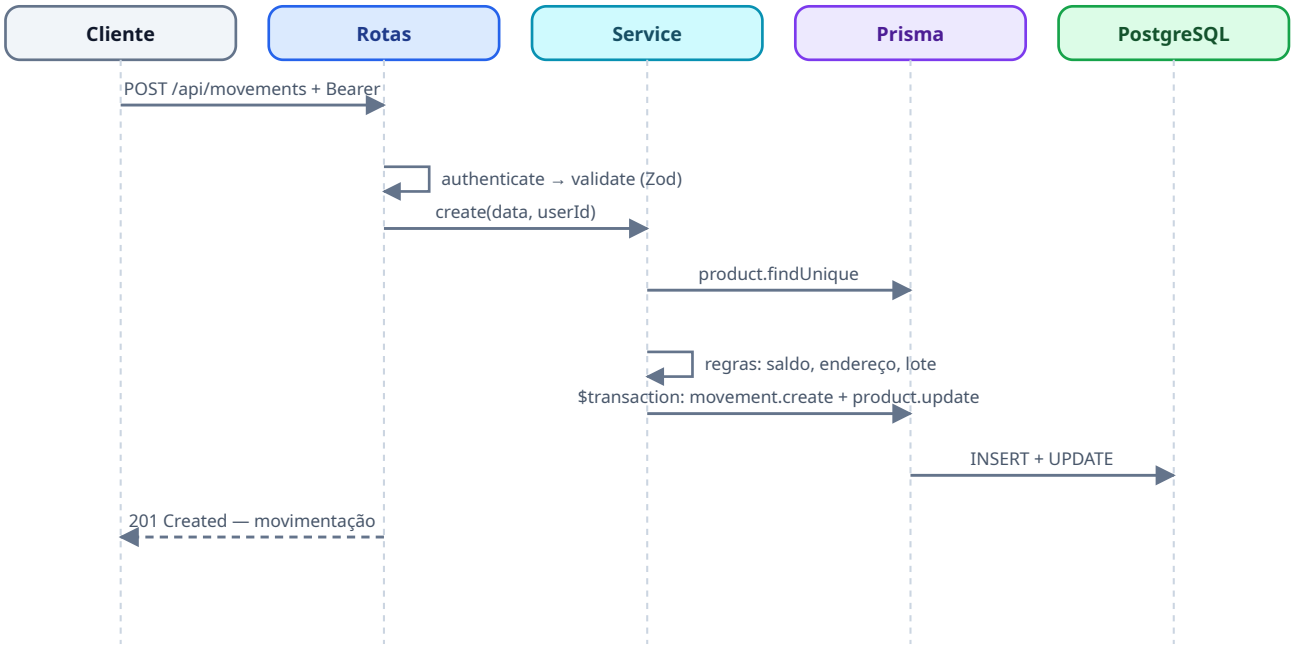


Figura 3 — POST /api/movements

05 Modelo de dados

O modelo é definido em `prisma/schema.prisma` e versionado por **5 migrations**. São **14 entidades** e **12 enumerações**; os identificadores são `cuid` e as tabelas usam nomes em `snake_case` plural.

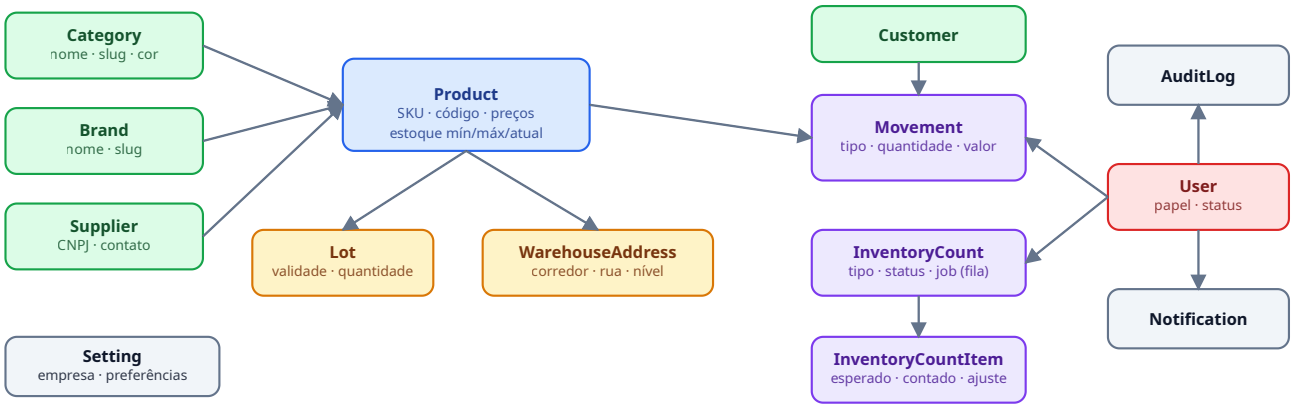


Figura 4 — Diagrama entidade-relacionamento (simplificado)

Entidades

Entidade	Campos principais	Relacionamentos
User	nome, e-mail (único), senha (hash), papel, departamento, status, avatarUrl, lastLogin	1:N Movement, AuditLog, InventoryCount, Notification
Category · Brand	nome, slug, cor / logoUrl	1:N Product

Entidade	Campos principais	Relacionamentos
Supplier · Customer	nome, nome fantasia, CNPJ (único), e-mail, telefone, contato, cidade, UF, status	Supplier: 1:N Product, Movement, Lot · Customer: referenciado por Movement
Product	nome, código interno, SKU, código de barras, unidade, dimensões, preços, estoque mín/máx/atual, status, `imageUrl`, `imageKey`	N:1 Category, Brand, Supplier; 1:N Movement, Lot, WarehouseAddress
WarehouseAddress	código, corredor, rua, prateleira, nível, posição, status, capacidade, ocupação, lote, quantidade	N:1 Product (opcional); origem/destino de Movement
Movement	tipo, quantidade, custo, valor total, NF, lote, motivo de saída, observações	N:1 Product, User; opcionais Supplier, Customer, Lot, endereços
Lot	número (único), quantidade, fabricação, validade, endereço, status	N:1 Product, Supplier
InventoryCount	nome, tipo, status, datas, totais, **jobStatus**, jobError, jobQueuedAt, jobFinishedAt, adjustedItems, failedItems	N:1 User (responsável); 1:N InventoryCountItem
InventoryCountItem	esperado, contado, divergência, contado por/em, **adjustedAt**, **adjustmentError**	N:1 InventoryCount, Product, User
Notification	alertKey, readAt — único por (alertKey, userId)	N:1 User
AuditLog	ação, entidade, id, nome, valor antigo/novo (JSON), IP	N:1 User
Setting	chave (`company` , `preferences`) e valor JSON	—

Enumerações

Enum	Valores
UserRole	admin · supervisor · operator · viewer
UserStatus	active · inactive · pending
ProductStatus	active · inactive · discontinued
PositionStatus	free · occupied · blocked · reserved
MovementType	entry · exit · transfer · loss · adjustment · inventory
ExitReason	sale · transfer · loss · break · internal
LotStatus	valid · expiring · expired · quarantine
InventoryCountType	full · partial · cyclic
InventoryCountStatus	planned · in_progress · review · completed
InventoryJobStatus	pending · processing · completed · partial · failed
SupplierStatus · CustomerStatus	active · inactive

06 Autenticação e autorização

O login retorna um token JWT assinado com `JWT_SECRET`, contendo `sub` (id), `email` e `role`. Rotas protegidas exigem `Authorization: Bearer <token>`. Senhas são armazenadas apenas como hash bcrypt. O middleware `authenticate` confere o token e o estado/papel atuais da conta; `authorize(...)` aplica a lista de papéis permitidos em cada rota.

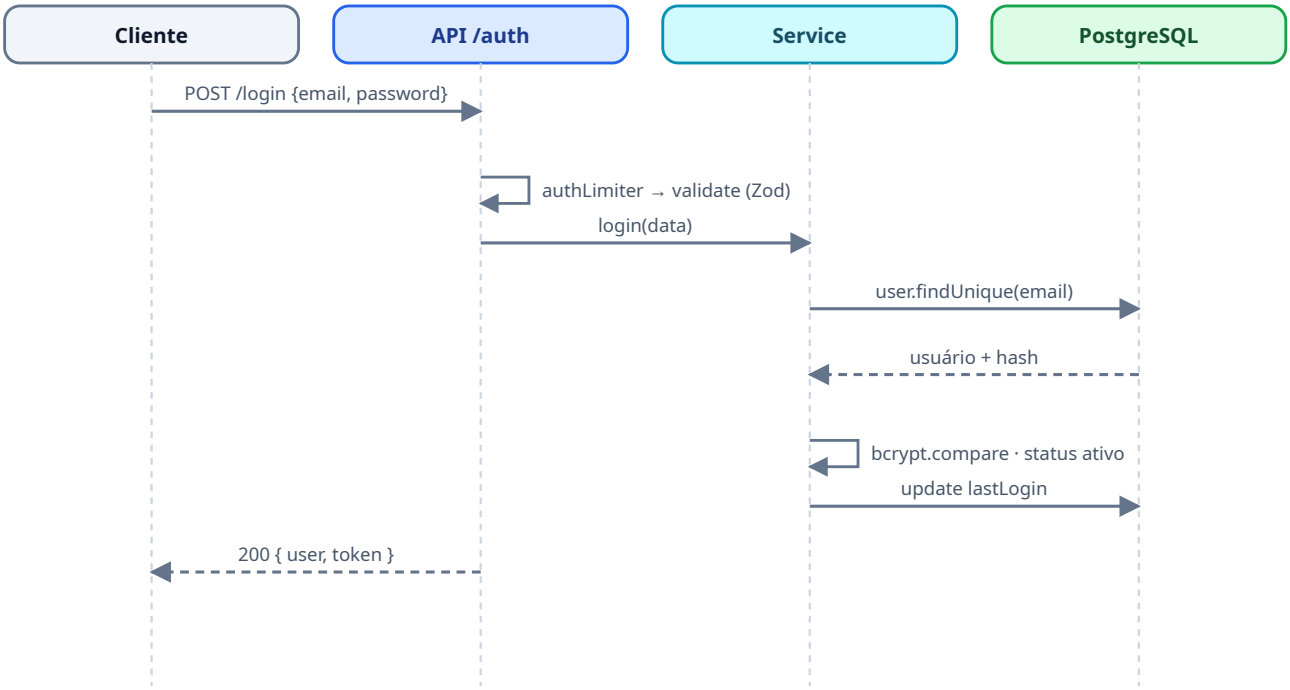


Figura 5 — Fluxo de login

COMO É APLICADO

As permissões são configuradas **explicitamente em cada rota**: a lista de papéis de `authorize(...)` é a fonte de verdade (visível no Swagger, em "Perfis permitidos"). Falhas retornam 401 (token) ou 403 (permissão). Os rótulos abaixo (Supervisor+, Operador+) apenas resumem as listas usadas.

Módulo	Leitura	Criar	Alterar	Excluir
Produtos · Categorias · Marcas · Fornecedores · Clientes · Endereços	Autenticado	admin, supervisor	admin, supervisor	admin
Foto do produto (upload/remoção)	Autenticado (URL assinada)	admin, supervisor	admin, supervisor	admin, supervisor
Usuários	Autenticado	admin, supervisor	admin, supervisor (status e senha: admin)	admin
Movimentações	Autenticado	admin, supervisor, operator	—	—

Módulo	Leitura	Criar	Alterar	Excluir
Lotes	Autenticado	admin, supervisor, operator	admin, supervisor	admin
Inventário (contagens)	Autenticado	admin, supervisor	admin, supervisor, operator (reprocessar: admin, supervisor)	admin
Dashboard · Relatórios · Analytics	Autenticado	—	—	—
Alertas	Autenticado	—	marcar como lido: autenticado	—
Auditoria	admin, supervisor	—	—	—
Configurações	Autenticado	—	admin	—
Auth: register · login	Público	—	senha/perfil: autenticado	—

07 Regras de negócio

Registro de movimentações

A criação de uma movimentação é atômica: o registro do movimento e a atualização do saldo (produto, endereço e lote) ocorrem na mesma transação. `entry` e `adjustment` (aumento) incrementam o saldo; `exit`, `loss` e `adjustment` (redução) decrementam e exigem saldo suficiente (senão 400); `inventory` define o saldo para a quantidade contada; `transfer` move entre endereços sem alterar o saldo. O valor total é `quantity × unitCost`. A lógica é exposta como `createWithin(tx, …)` para ser reaproveitada em outros fluxos transacionais (o worker do inventário).

Situação de estoque

Situação	Regra (saldo × mínimo)	Uso
ok	saldo > mínimo	Listagem de produtos e filtros
low	mínimo/2 < saldo ≤ mínimo	Alerta de reposição
critical	0 < saldo ≤ mínimo/2	Alerta de alta prioridade
out	saldo = 0	Alerta de ruptura

Inventário e ajuste de estoque assíncrono

A contagem segue `planned → in_progress → review → completed`. Registrar a contagem de um item calcula a divergência (`contado − esperado`). Ao **concluir** (`review → completed`), a API grava `jobStatus = pending` e publica uma mensagem no **SQS**; o **worker** então registra uma movimentação `inventory` para

cada item divergente (o estoque passa a ser a quantidade contada), em lotes de `INVENTORY_BATCH_SIZE` itens, atualizando `adjustedItems` / `failedItems` para a tela acompanhar.

- **Idempotência:** cada item só é ajustado se `adjustedAt` for nulo — marca gravada na mesma transação da movimentação. Reentregas e reprocessamentos não duplicam ajustes.
- **Pendência manual:** produto em vários endereços ou lotes não pode ser ajustado automaticamente; o item recebe `adjustmentError` e o job termina como `partial`.
- **Falhas:** erro inesperado devolve a mensagem à fila; após `QUEUE_MAX_RECEIVES` tentativas o job vira `failed` e a fila move a mensagem à DLQ. `POST /api/inventory/{id}/reprocess` reenfileira jobs `failed`, `partial` ou travados há mais de 15 min.
- Inventários concluídos **antes** deste recurso têm `jobStatus = null` e não são reprocessados.

Foto de produto (Amazon S3)

`POST /api/products/{id}/image` recebe `multipart/form-data` (campo `image`; JPEG, PNG, WebP ou GIF até 5 MB), confere o tipo pelos primeiros bytes (SVG é recusado), grava no S3 em `<prefixo>products/<idProduto>/<uuid>.<ext>` e guarda a chave em `products.imageKey`. As respostas de produto trazem `imageUrl` como **URL assinada temporária**. A foto antiga só é apagada depois que a nova está gravada; excluir o produto apaga a foto. `npm run storage:cleanup` remove objetos órfãos. Detalhes em *Computação em Nuvem*.

Alertas, lotes e curva ABC

O módulo de alertas consolida, em tempo real, produtos em ruptura, críticos e com estoque baixo, além de lotes a vencer (janela configurável, padrão 30 dias) e vencidos; a leitura é persistida por usuário em `notifications` (`alertKey`). Lotes têm número único, fabricação, validade, endereço e status. A curva ABC ordena os produtos pelo valor movimentado (`Σ totalValue`): classe A até 80% acumulado, B até 95%, C acima.

Analytics (previsão de demanda e matriz ABC×XYZ)

`GET /api/analytics` entrega a previsão semanal de demanda (histórico real × projeção, com faixa de incerteza), a matriz **ABC × XYZ** (valor movimentado × variabilidade da demanda, CV = desvio ÷ média: X < 0,5; Y até 1,0; Z acima), o giro e as **sugestões de compra** (cobertura de 28 dias de demanda prevista + estoque de segurança, considerando o prazo de reposição das configurações) com urgência e motivo, além de *insights*. A previsão vem de um serviço de ML **opcional** (`ML_SERVICE_URL`); sem ele — que é o caso deste repositório, que não inclui o serviço Python — o backend usa **média móvel**, e `GET /api/analytics/model` informa a origem usada.

Dashboard e configurações

O dashboard oferece resumo geral (KPIs, ocupação e valor de estoque), tendência mensal de entradas e saídas, distribuição por categoria, produtos mais movimentados, curva ABC e mapa de calor por dia/hora. `GET/PUT /api/settings` guarda os dados da empresa e as preferências (dias de alerta de validade e prazo de reposição); só o administrador altera.

08 Catálogo de endpoints

Base URL padrão: `http://localhost:3001`. Todas as rotas ficam sob `/api`. Listagens aceitam paginação (`page`, `limit` ≤ 100) e retornam `{ data, total, page, limit, totalPages }`. Esta tabela é gerada a partir do `swagger.json` exportado do código (`npm run docs:export`).

01 · AUTENTICAÇÃO /API/AUTH

Método	Rota	Descrição	Acesso
GET	`/me`	Dados do usuário autenticado	Autenticado
GET	`/profile`	Perfil completo	Autenticado
POST	`/register`	Registrar usuário (papel padrão: operador)	Público
POST	`/login`	Login e emissão do token	Público
PATCH	`/profile`	Atualizar o próprio perfil	Autenticado
PATCH	`/password`	Alterar a própria senha	Autenticado

02 · USUÁRIOS /API/USERS

Método	Rota	Descrição	Acesso
GET	`/`	Listar (paginado)	Autenticado
GET	`/:id`	Buscar por id	Autenticado
POST	`/`	Criar usuário	Supervisor+
PATCH	`/:id`	Atualizar usuário	Supervisor+
DELETE	`/:id`	Remover usuário	Admin
PATCH	`/:id/status`	Alterar status	Admin
PATCH	`/:id/password`	Redefinir a senha de um usuário	Admin

03 · PRODUTOS /API/PRODUCTS

Método	Rota	Descrição	Acesso
GET	`/`	Listar com busca, categoria, marca, status e situação de estoque	Autenticado
GET	`/:id`	Detalhe com situação de estoque	Autenticado
GET	`/scan/:code`	Buscar por código de barras / SKU / código interno	Autenticado
GET	`/`	Servir arquivo do driver local por URL assinada	Público
POST	`/`	Criar produto (SKU e código únicos)	Supervisor+
PATCH	`/:id`	Atualizar produto	Supervisor+
DELETE	`/:id`	Remover produto (e a foto no storage)	Admin

Método	Rota	Descrição	Acesso
POST	`/:id/image`	Enviar a foto do produto (multipart, campo `image`) para o S3	Supervisor+
DELETE	`/:id/image`	Remover a foto do produto	Supervisor+

04 · CATEGORIAS /API/CATEGORIES

Método	Rota	Descrição	Acesso
GET	`/`	Listar	Autenticado
GET	`/:id`	Buscar por id	Autenticado
POST	`/`	Criar	Supervisor+
PATCH	`/:id`	Atualizar	Supervisor+
DELETE	`/:id`	Remover	Admin

05 · MARCAS /API/BRANDS

Método	Rota	Descrição	Acesso
GET	`/`	Listar	Autenticado
GET	`/:id`	Buscar por id	Autenticado
POST	`/`	Criar	Supervisor+
PATCH	`/:id`	Atualizar	Supervisor+
DELETE	`/:id`	Remover	Admin

06 · FORNECEDORES /API/SUPPLIERS

Método	Rota	Descrição	Acesso
GET	`/`	Listar (paginado)	Autenticado
GET	`/:id`	Buscar por id	Autenticado
POST	`/`	Criar	Supervisor+
PATCH	`/:id`	Atualizar	Supervisor+
DELETE	`/:id`	Remover	Admin

07 · CLIENTES /API/CUSTOMERS

Método	Rota	Descrição	Acesso
GET	/	Listar com busca e paginação	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

08 · ARMAZÉM (ENDEREÇOS) /API/WAREHOUSE

Método	Rota	Descrição	Acesso
GET	/	Listar endereços (filtros)	Autenticado
GET	/stats	Estatísticas de ocupação	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar endereço	Supervisor+
PATCH	/:id	Atualizar	Supervisor+
DELETE	/:id	Remover	Admin

09 · MOVIMENTAÇÕES /API/MOVEMENTS

Método	Rota	Descrição	Acesso
GET	/	Listar por tipo, produto e período	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Registrar entrada, saída, transferência, perda, ajuste ou inventário	Operador+

10 · LOTES /API/LOTS

Método	Rota	Descrição	Acesso
GET	/	Listar (produto, status, vencimento)	Autenticado
GET	/alerts	Lotes a vencer / vencidos	Autenticado
GET	/:id	Buscar por id	Autenticado
POST	/	Criar lote (número único)	Operador+
PATCH	/:id	Atualizar lote	Supervisor+
DELETE	/:id	Remover lote	Admin

11 · INVENTÁRIO /API/INVENTORY

Método	Rota	Descrição	Acesso
GET	/	Listar contagens (com situação do ajuste de estoque)	Autenticado
GET	/:id	Detalhe da contagem e dos itens	Autenticado
POST	/	Planejar contagem	Supervisor+
PATCH	/:id	Atualizar andamento; ao concluir, enfileira o ajuste de estoque	Operador+
DELETE	/:id	Remover contagem planejada	Admin
POST	/:id/items/:itemId/count	Registrar a quantidade contada de um item	Operador+
POST	/:id/reprocess	Reenfileirar o ajuste de estoque (falhou, parcial ou travado)	Supervisor+

12 · DASHBOARD /API/DASHBOARD

Método	Rota	Descrição	Acesso
GET	/	Resumo geral	Autenticado
GET	/movement-trend	Tendência mensal	Autenticado
GET	/category-distribution	Distribuição por categoria	Autenticado
GET	/top-products	Mais movimentados	Autenticado
GET	/abc	Curva ABC	Autenticado
GET	/heatmap	Volume por dia da semana e hora	Autenticado

13 · ALERTAS /API/ALERTS

Método	Rota	Descrição	Acesso
GET	/	Todos os alertas com estado de leitura	Autenticado
GET	/stock	Alertas de estoque por severidade	Autenticado
GET	/expiring	Lotes próximos ao vencimento	Autenticado
PATCH	/read-all	Marcar todos como lidos	Autenticado
PATCH	/:id/read	Marcar um alerta como lido	Autenticado

14 · AUDITORIA /API/AUDIT

Método	Rota	Descrição	Acesso
GET	/	Logs de auditoria	Supervisor+
GET	/:id	Detalhe de um log	Supervisor+

15 · RELATÓRIOS /API/REPORTS

Método	Rota	Descrição	Acesso
GET	/movements	Movimentações por tipo e produto	Autenticado
GET	/stock	Posição de estoque valorizada	Autenticado
GET	/lots	Validade de lotes	Autenticado
GET	/warehouse	Ocupação do armazém	Autenticado
GET	/abc	Curva ABC	Autenticado
GET	/inventory	Contagens e divergências	Autenticado
GET	/suppliers	Compras por fornecedor	Autenticado

16 · ANALYTICS (IA) /API/ANALYTICS

Método	Rota	Descrição	Acesso
GET	/	Previsão de demanda, matriz ABC×XYZ, sugestões de compra e insights	Autenticado
GET	/model	Situação e métricas do modelo de previsão	Autenticado

17 · CONFIGURAÇÕES /API/SETTINGS

Método	Rota	Descrição	Acesso
GET	/	Dados da empresa e preferências do sistema	Autenticado
PUT	/	Atualizar empresa e/ou preferências	Admin

09 Validação e tratamento de erros

Cada módulo possui schemas Zod em `src/schemas`. O middleware `validate` valida o corpo e `validateQuery` valida e converte (`coerce`) os parâmetros de consulta antes de chegarem ao controller.

```
const createMovementSchema = z.object({
  type: z.enum(['entry', 'exit', 'transfer', 'loss', 'adjustment', 'inventory']),
  productId: z.string().cuid(),
  quantity: z.number().int().min(0),
  unitCost: z.number().min(0).default(0),
  ...
}).superRefine(/* regras cruzadas: quantidade > 0, direção do ajuste, endereços da transferência
*/)
```

Código	Origem	Corpo
400	Regra de negócio violada (ex.: estoque insuficiente, arquivo ausente)	`{ message }`
401	Token ausente, inválido ou expirado; credenciais inválidas	`{ message }`

Código	Origem	Corpo
403	Papel sem permissão; conta inativa	`{ message }`
404	Recurso não encontrado	`{ message }`
409	Conflito (unicidade, estado concorrente, nada a reprocessar)	`{ message }`
413	Foto maior que `STORAGE_MAX_IMAGE_MB`	`{ message }`
415	Arquivo que não é JPEG, PNG, WebP ou GIF	`{ message }`
422	Falha de validação Zod	`{ message, errors: [{ field, message }] }`
429	Limite de requisições excedido	`{ message }`
500	Erro inesperado (detalhes só no log do servidor)	`{ message }`
503	Storage ou fila não configurados/indisponíveis; banco indisponível	`{ message }`

10 Segurança e auditoria

Mecanismo	Detalhes
Helmet	Cabeçalhos HTTP de segurança em toda a API
CORS	Habilitado para consumo pelo front-end (revisar origens por ambiente antes de implantar)
Rate limiting	`globalLimiter`: 200 req / 15 min por IP · `authLimiter`: 20 req / 15 min em `/auth/login` e `/auth/register`
Senhas	Hash bcrypt (custo 10); o hash nunca é devolvido nas respostas
JWT	Assinado com `JWT_SECRET`, expiração configurável (`JWT_EXPIRES_IN`)
Validação	Toda entrada passa por schema Zod antes da regra de negócio
Upload	Autenticado, papel admin/supervisor, limite de tamanho, tipo conferido pelo conteúdo, SVG recusado, chaves restritas ao prefixo do projeto
Credenciais de nuvem	Apenas em variáveis de ambiente (`.env` fora do Git); URLs assinadas têm validade curta
Erros	`AppError` + handler central; erros internos não vazam detalhes ao cliente

A tabela `audit_logs` registra ação, entidade, id e nome, valor novo/antigo (JSON), IP e usuário. O middleware `audit(entity, action)` intercepta a resposta JSON de operações bem-sucedidas e grava o log sem atrasar a resposta; senhas e **URLs assinadas** são removidas antes de gravar. A consulta fica em `/api/audit`, restrita a supervisor e administrador.

11 Testes automatizados

A suíte (Jest + Supertest) exercita a aplicação Express real contra um PostgreSQL dedicado (`TEST_DATABASE_URL`, com "test" no nome) e usa simulações para AWS. Última execução completa: **173 de 173 passando** (19/10/2026).

Arquivo	Escopo	Casos
`audit-sanitize.test.ts`	Remoção de segredos no log de auditoria	1
`auth.test.ts`	Registro, login, perfil e senha	9
`category.test.ts`	CRUD de categorias	9
`critical-security.test.ts`	Segurança: acesso, tokens, validações críticas	10
`flows.test.ts`	Fluxos ponta a ponta entre módulos	21
`inventory-async-e2e.test.ts`	Inventário assíncrono ponta a ponta (banco real)	4
`inventory-processing.test.ts`	Ajuste de estoque: idempotência, lotes, pendências	10
`inventory-queue-flow.test.ts`	Concluir inventário enfileira; reprocessamento	8
`inventory-service.test.ts`	Transições e contagem do inventário	3
`inventory-worker.test.ts`	Worker: retry, falha final, mensagem inválida	5
`ml-analytics.test.ts`	Analytics, ABC×XYZ e sugestões	4
`ml-service.test.ts`	Cliente do serviço de previsão e fallback	6
`movement-service.test.ts`	Regras de movimentação (serviço)	6
`movement.test.ts`	Entrada/saída e saldo	5
`product-image-service.test.ts`	Foto de produto: ordem, falhas, limites, URL assinada	12
`product.test.ts`	CRUD de produtos, scan, filtros	8
`queue-service.test.ts`	Fila SQS/local: envio, visibilidade, DLQ	10
`queue-sqs-integration.test.ts`	Driver SQS contra servidor SQS real (opcional)	2
`report-service.test.ts`	Relatórios	1
`storage-service.test.ts`	Storage S3/local, URLs assinadas, tipos de imagem	16
`user.test.ts`	CRUD de usuários e permissões	15
`warehouse.test.ts`	Endereços de armazém	8
Total		**173**

```
npm test                # suíte completa (--runInBand)
npm run test:coverage   # relatório em coverage/
SQS_TEST_ENDPOINT=http://127.0.0.1:9324 npm test -- queue-sqs-integration   # teste opcional do driver SQS
```

12 Operação e execução

```
cd backend
npm install
cp .env.example .env          # ajuste DATABASE_URL, JWT_SECRET (+ storage/fila, se usar)
npm run prisma:migrate        # aplica migrations
npm run prisma:seed           # dados de demonstração
npm run dev                   # http://localhost:3001 (hot-reload; worker embutido)
```

Variáveis de ambiente

Variável	Descrição
`DATABASE_URL` · `TEST_DATABASE_URL`	PostgreSQL de aplicação e de testes (nome com "test")
`PORT` · `NODE_ENV`	Porta (3001) e ambiente
`JWT_SECRET` · `JWT_EXPIRES_IN`	Assinatura e validade dos tokens
`STORAGE_DRIVER`, `S3_*`, `STORAGE_*`	Fotos de produto: driver `s3` / `local`, bucket, região, credenciais, prefixo, validade das URLs, limite de tamanho, carência de órfãos
`QUEUE_DRIVER`, `SQS_*`, `QUEUE_*`, `INVENTORY_BATCH_SIZE`	Mensageria: driver `sqs` / `local`, URL da fila, credenciais, visibilidade, tentativas, worker `embedded` / `off`, tamanho do lote
`ML_SERVICE_URL` · `ML_TIMEOUT_MS`	Serviço de previsão opcional; sem ele a previsão usa média móvel

Scripts

Script	Função
`dev` · `build` · `start`	Hot-reload; compila para `dist/`; executa o build
`worker` · `worker:start`	Processo dedicado ao consumo da fila (use `QUEUE_WORKER=off` na API)
`prisma:migrate` · `prisma:generate` · `prisma:studio` · `prisma:seed`	Migrations, client, interface gráfica e massa de demonstração
`queues:setup`	Cria a fila de inventário e a DLQ no SQS e imprime as variáveis
`storage:cleanup`	Remove do storage fotos sem produto (`--dry-run` só lista)
`docs:check` · `docs:export`	Confere rotas × Swagger · exporta `docs/swagger.json`
`test` · `test:coverage`	Testes e cobertura

URL	Conteúdo
`GET /`	Painel HTML com todos os módulos e endpoints
`GET /docs`	Swagger UI com autenticação Bearer
`GET /docs.json`	Especificação OpenAPI 3.0
`GET /health/live` · `/health/ready`	Liveness e prontidão (conexão com o banco)

13 Infraestrutura em nuvem

Os dois serviços AWS usados pelo backend estão detalhados no documento **Computação em Nuvem II**: Amazon S3 (fotos) e Amazon SQS (fila do inventário). A arquitetura-alvo prevê API e worker em EC2, banco em RDS, Application Load Balancer, ACM e CloudWatch.

Serviço	Papel	Situação
Amazon S3	Fotos de produto (upload via API, URLs assinadas)	**Implementado**
Amazon SQS	Fila de inventário + DLQ, consumida pelo worker	**Implementado · fila real pendente**
EC2 · ALB · RDS · CloudWatch	Hospedagem, balanceamento, banco e observabilidade	**Planejado**